

Learning rate scheduling

(управление скоростью обучения)

Lambda LR I

Задает lr равным начальному lr , умноженному на заданную функцию.

$$lr_{\text{epoch}} = lr_{\text{initial}} * \text{Lambda}(\text{epoch}) \quad (1)$$

```
model = torch.nn.Linear(2, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=100)
lambda1 = lambda epoch: 0.65 ** epoch
scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer,
        lr_lambda=lambda1)
lrs = []
for i in range(10):
    optimizer.step()
    lrs.append(optimizer.param_groups[0]["lr"])
    scheduler.step()
```

Lambda LR II

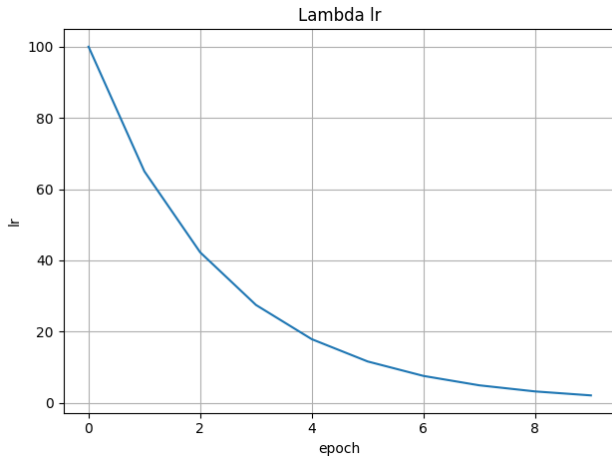


Рис. 1 – Lambda lr

MultiplicativeLR I

Умножает lr с предыдущего шага на заданную функцию.

$$lr_{\text{epoch}} = lr_{\text{epoch} - 1} * \text{Lambda}(\text{epoch}) \quad (2)$$

```
lmbda = lambda epoch: 0.65 ** epoch
scheduler = torch.optim.lr_scheduler.MultiplicativeLR(
    optimizer, lr_lambda=lmbda)
lrs = []
for i in range(10):
    optimizer.step()
    lrs.append(optimizer.param_groups[0]["lr"])
    scheduler.step()
```

MultiplicativeLR II

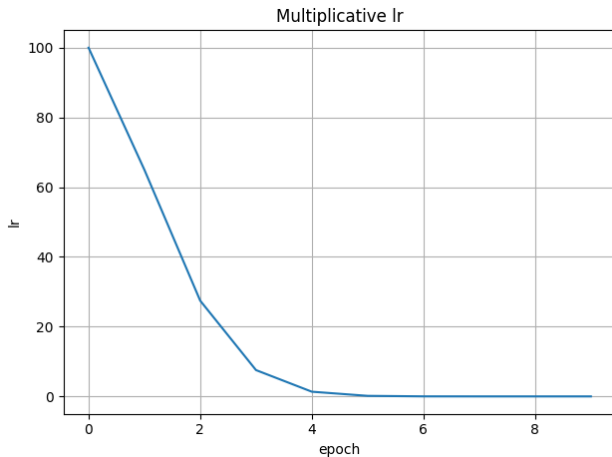


Рис. 2 – Multiplicative lr

StepLR I

Уменьшение lr в γ каждые `step_size` шагов.

$$lr_{\text{epoch}} = \begin{cases} \gamma \cdot lr_{\text{epoch}-1}, & \text{если epoch \% step_size} = 0, \\ lr_{\text{epoch}-1}, & \text{иначе.} \end{cases} \quad (3)$$

```
scheduler = torch.optim.lr_scheduler.StepLR(optimizer,  
    step_size=2,  
    gamma=0.5)
```

StepLR II

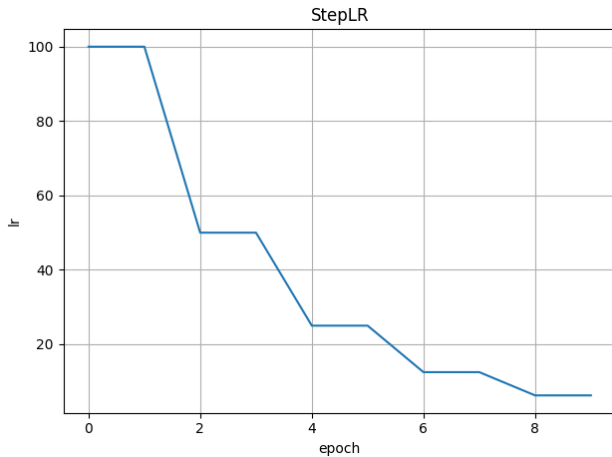


Рис. 3 – StepLR

MultiStepLR I

Уменьшение lr в γ при достижении контрольных точек для количества эпох (milestones).

$$lr_{\text{epoch}} = \begin{cases} \gamma \cdot lr_{\text{epoch}-1}, & \text{if epoch in [milestones]}, \\ lr_{\text{epoch}-1}, & \text{иначе.} \end{cases} \quad (4)$$

```
scheduler = torch.optim.lr_scheduler.MultiStepLR(  
    optimizer, milestones=[6,8,9],  
    gamma  
        =0.1)
```


MultiStepLR II

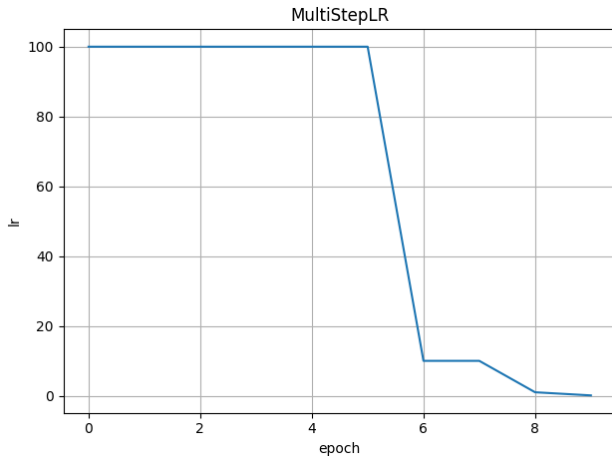


Рис. 4 – StepLR

ExponentialLR I

Уменьшение lr в γ на каждой эпохе.

$$lr_{\text{epoch}} = \text{Gamma} * lr_{\text{epoch} - 1} \quad (5)$$

```
scheduler = torch.optim.lr_scheduler.ExponentialLR(  
    optimizer, gamma=0.5)
```

ExponentialLR II

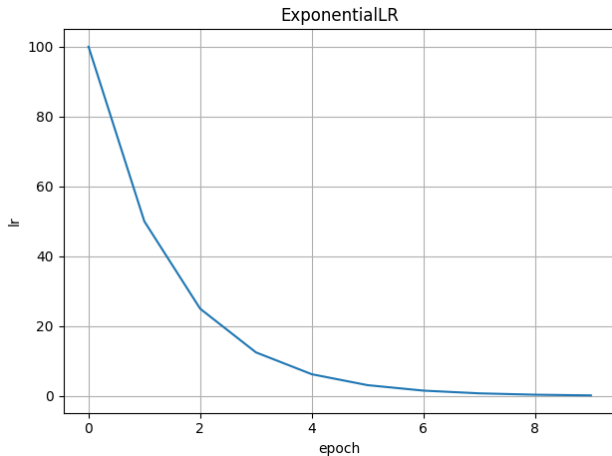


Рис. 5 – ExponentialLR

CosineAnnealingLR I

Уменьшение lr по закону $\cos$:

$$\eta_t = \eta_{\min} + \frac{1}{2} (\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{T_{cur}}{T_{\max}} \pi \right) \right) \quad (6)$$

Был предложен в SGDR: Stochastic Gradient Descent with Warm Restarts. <https://arxiv.org/abs/1608.03983>

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=10, eta_min=0)
```

CosineAnnealingLR II

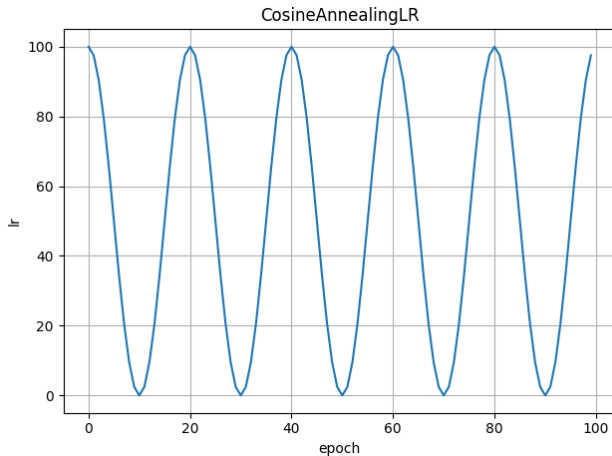


Рис. 6 – CosineAnnealingLR

CyclicLR I

```
scheduler = torch.optim.lr_scheduler.CyclicLR(optimizer,  
    base_lr=0.001, max_lr=0.1,step_size_up=5,  
    mode="triangular")
```

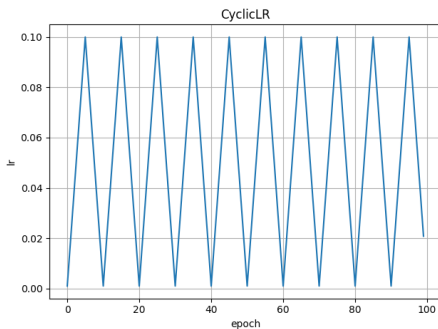


Рис. 7 – CyclicLR

CyclicLR - triangular2 I

```
scheduler = torch.optim.lr_scheduler.CyclicLR(optimizer,  
    base_lr=0.001, max_lr=0.1,step_size_up=5,  
    mode="triangular2")
```

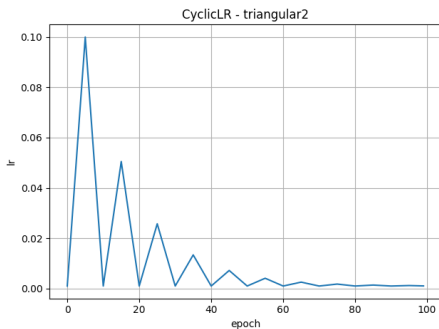


Рис. 8 – CyclicLR - triangular2

CyclicLR - exp range I

```
scheduler = torch.optim.lr_scheduler.CyclicLR(optimizer,  
    base_lr=0.001, max_lr=0.1, step_size_up=5,  
    mode="exp_range", gamma=0.85)
```

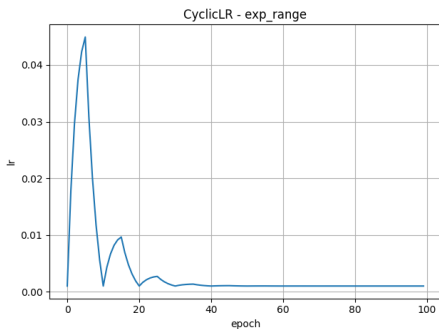
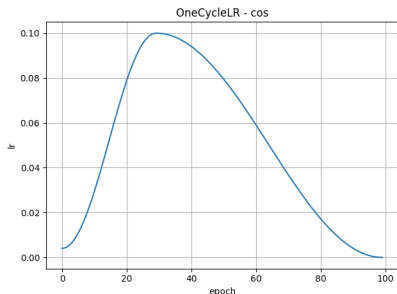


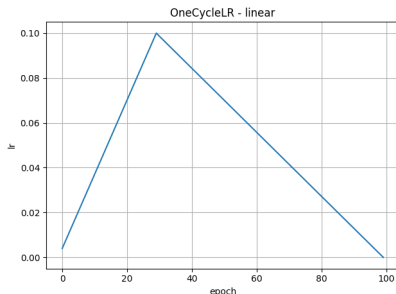
Рис. 9 – CyclicLR - exp range

OneCycleLR I

Обновление lr происходит в одном цикле: от максимума до минимума по определённому правилу (cos или линейному).



(a) Косинусное изменение.
`scheduler = torch.optim.lr_scheduler.
OneCycleLR(optimizer, max_lr=0.1,
steps_per_epoch=10, epochs=10)`



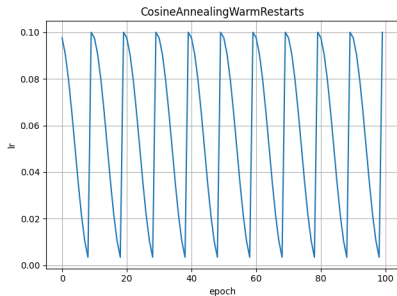
(b) Линейное изменение. `scheduler =
torch.optim.lr_scheduler.OneCycleLR(
optimizer, max_lr=0.1, steps_per_epoch=10,
epochs=10, anneal_strategy='linear')`

CosineAnnealingWarmRestarts I

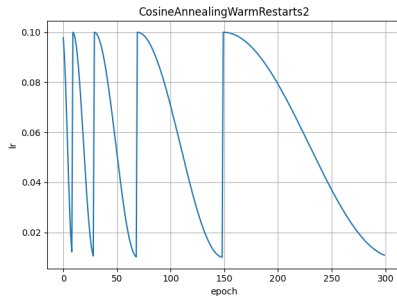
Обновление lr происходит, как в cos-annealing schedule, и повторяется каждые T_i эпох.

$$\eta_t = \eta_{\min} + \frac{1}{2} (\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{T_{\text{cur}}}{T_i} \pi \right) \right) \quad (7)$$

CosineAnnealingWarmRestarts II



(a) `lr_sched = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(optimizer, T_0=10, T_mult=1, eta_min=0.001, last_epoch=-1)`



(b) `lr_sched = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(optimizer, T_0=10, T_mult=2, eta_min=0.01, last_epoch=-1)`